

ZONE NAV SOFTWARE DESIGN GUIDE V1.0

Zone-Aware Navigation Software Design Guide

diy_zone_nav • cmd_vel_mux • lidar_odom_gate

Distilled from 46 bugs — hard-won implementation knowledge for the 2026 DIY Robot Challenge.

COMPETITION

October 1, 2026

PACKAGES

diy_zone_nav

PLATFORM

Jetson Orin Nano

TEAM

Juggernauts 2026

ROS 2 Humble

zone_nav_manager

cmd_vel_mux

lidar_odom_gate

7-State Machine

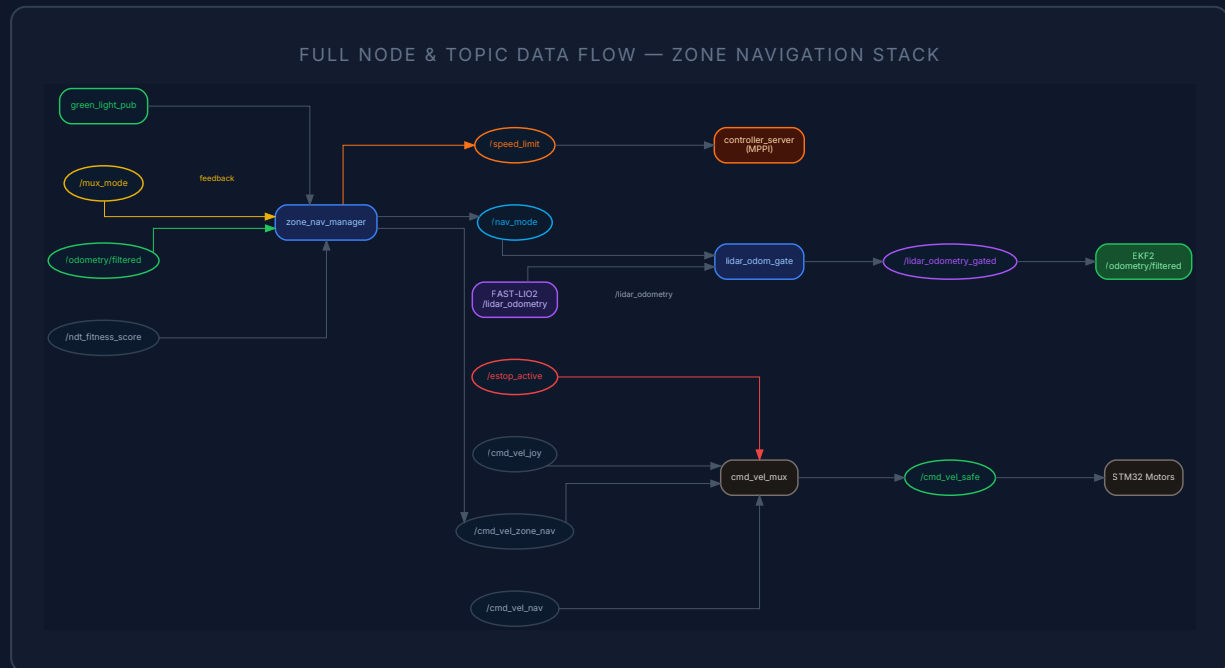
46 Bugs Documented

zone_waypoints.yaml

Table of Contents

- 1 System Architecture
- 2 State Machine Reference
- 3 Topic & Service Contracts
 - 3.1 zone_nav_manager Publishers
 - 3.2 zone_nav_manager Subscribers
 - 3.3 cmd_vel_mux Publishers / Subscribers
 - 3.4 SetParameters Services
- 4 Parameter Reference
- 5 zone_waypoints.yaml Format
- 6 Key Implementation Gotchas
 - 6.1 Speed Control API
 - 6.2 Costmap Service Paths — The Doubled Namespace
 - 6.3 VoxelLayer Limitations
 - 6.4 ESTOP_LOCK is Hardware-Only
 - 6.5 BLIND_DRIVE Must-Knows
 - 6.6 zone_waypoints.yaml Pitfalls
 - 6.7 lidar_odom_gate Must Always Run with use_localization
- 7 Debugging Checklist
- 8 Scripts Reference

The zone navigation stack centres on three cooperating nodes. **zone_nav_manager** monitors robot position and NDT fitness, drives the 7-state machine, and publishes control signals. **cmd_vel_mux** arbitrates between joystick, Nav2, and blind-drive velocity sources. **lidar_odom_gate** gates the FAST-LIO2 odometry stream to EKF2, suppressing lidar during BLIND_DRIVE to prevent drift injection.



zone_nav_manager

The brain. Subscribes to position (`/odometry/filtered`) and NDT fitness score. Checks each registered waypoint zone on every timer tick. Publishes `/nav_mode`, `/speed_limit`, and sets costmap + mux parameters via service calls.

cmd_vel_mux

Velocity arbitrator. Accepts `/cmd_vel_joy`, `/cmd_vel_nav`, and `/cmd_vel_zone_nav`. Selects which source reaches `/cmd_vel_safe` based on mode and `/estop_active`. ESTOP_LOCK overrides all sources.

lidar_odom_gate

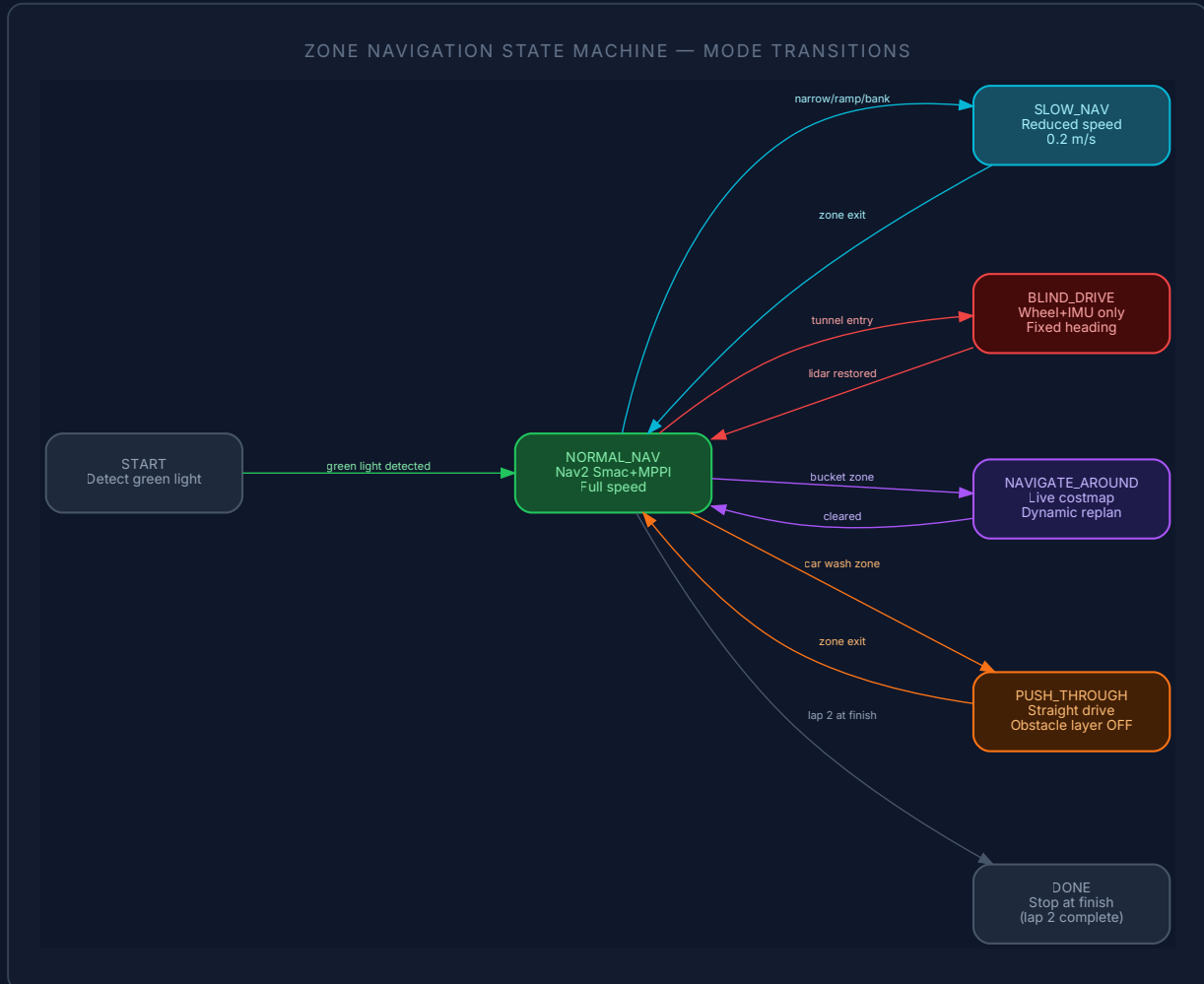
Inline gate on `/lidar_odometry` (FAST-LIO2). Passes through when `/nav_mode` $\neq$ BLIND_DRIVE; blocks when BLIND_DRIVE is active. Prevents corrupt lidar odometry from poisoning EKF2 inside the tunnel. 5s watchdog re-opens on manager crash.

KEY DEPENDENCY CHAIN

green_light_pub → zone_nav_manager → /nav_mode → lidar_odom_gate → EKF2 → /odometry/filtered → zone_nav_manager (position feedback). Any break in this chain stalls the whole stack. The debugging checklist in Section 7 walks each link.

State Machine Reference

The zone_nav_manager implements a 7-state machine. Transitions are triggered by zone entry (distance from waypoint < radius) and exit (distance > exit_radius). DONE is triggered only after lap 2 is complete.



MODE	TRIGGER CONDITION	ENTRY ACTIONS	EXIT TRIGGER	EXIT ACTIONS	SPEED LIMIT	MUX MODE
INIT	Node startup, before green light	Gate open (pass-through). All params at defaults.	Green light detected on <code>/green_light</code>	Transition to NORMAL_NAV	No limit	JOYSTICK
NORMAL_NAV	Zone exit from any mode, or green light	Restore full speed. Enable obstacle layers. Set mux AUTONOMOUS.	Entry into a registered zone	Apply zone-specific mode	Configured max	AUTONOMOUS
SLOW_NAV	Narrow / ramp / bank zone entry	Publish <code>speed_limit=0.2</code> . Costmap unchanged.	Distance > exit_radius	Restore speed limit to 0.0 (=no limit)	0.2 m/s	AUTONOMOUS

BLIND_DRIVE

MODE	TRIGGER CONDITION	ENTRY ACTIONS	EXIT TRIGGER	EXIT ACTIONS	SPEED LIMIT	MUX MODE
BLIND_DRIVE	Tunnel entry zone	Set mux BLIND_DRIVE. Gate closes (blocks lidar to EKF2). Reset fitness to 999. 15s hard timeout armed.	NDT fitness < 0.8 after gate re- open, OR 15s timeout	Re-open gate. Restore mux AUTONOMOUS.	0.15 m/s fixed	
NAVIGATE_AROUND	Bucket zone entry	Enable obstacle_layer + voxel_layer. Full costmap active.	Distance > exit_radius	Restore NORMAL_NAV	0.3 m/s	AUTONOMOUS
PUSH_THROUGH	Car wash zone entry	Disable obstacle_layer.enabled AND voxel_layer.enabled. Straight heading cmd_vel.	Distance > exit_radius	Re-enable both layers. Restore NORMAL_NAV.	0.25 m/s	BLIND_DRIVE
DONE	Finish line zone, lap count == total_laps	Publish zero cmd_vel. Set mux JOYSTICK. Log completion time.	N/A — terminal state	N/A	0.0 (stop)	JOYSTICK

INIT KEEPS THE GATE OPEN

When zone_nav_manager is disabled or not yet started, the gate defaults to INIT (open). EKF2 receives full lidar odometry. This is intentional — safe pass-through for non-competition runs.

3.1 zone_nav_manager — Publishers

TOPIC	TYPE	QOS	PURPOSE
<code>/nav_mode</code>	<code>std_msgs/String</code>	Volatile, depth 10	Current state machine mode. Read by lidar_odom_gate and all debug tools.
<code>/speed_limit</code>	<code>std_msgs/Float32</code>	Volatile, depth 1	Speed limit to controller_server. Re-sent every 1s (reassertSpeedLimit). 0.0 = NO_LIMIT.
<code>/cmd_vel_zone_nav</code>	<code>geometry_msgs/Twist</code>	Volatile, depth 1	Direct velocity output during BLIND_DRIVE and PUSH_THROUGH modes.

3.2 zone_nav_manager — Subscribers

TOPIC	TYPE	PURPOSE
<code>/odometry/filtered</code>	<code>nav_msgs/Odometry</code>	Robot pose used for zone entry/exit distance checks.
<code>/ndt_fitness_score</code>	<code>std_msgs/Float32</code>	NDT scan-match quality. >0.8 = poor (tunnel). <0.8 = localized. Triggers BLIND_DRIVE exit.
<code>/green_light</code>	<code>std_msgs/Bool</code>	Race start trigger. True transitions INIT → NORMAL_NAV.
<code>/mux_mode</code>	<code>std_msgs/String</code>	Feedback from cmd_vel_mux. Used to verify mode changes landed correctly.

3.3 cmd_vel_mux — Publishers / Subscribers

Subscribers (velocity sources)

TOPIC	PRIORITY
<code>/cmd_vel_joy</code>	Highest (joystick override)
<code>/cmd_vel_nav</code>	Nav2 MPPI output
<code>/cmd_vel_zone_nav</code>	zone_nav_manager direct cmds

Publishers

TOPIC	PURPOSE
<code>/cmd_vel_safe</code>	Selected velocity → motors

TOPIC	PURPOSE
<code>/mux_mode</code>	Current active mode string

3.4 SetParameters Services Called by zone_nav_manager

SERVICE PATH	WHEN CALLED	PARAMETER SET
<code>/global_costmap/global_costmap/ set_parameters</code>	PUSH_THROUGH entry/exit	<code>obstacle_layer.enabled</code> , <code>inflation_layer.inflation_radius</code>
<code>/local_costmap/local_costmap/ set_parameters</code>	PUSH_THROUGH entry/exit	<code>voxel_layer.enabled</code> , <code>obstacle_layer.enabled</code>
<code>/cmd_vel_mux_node/set_parameters</code>	Mode transitions	<code>mode</code> (JOYSTICK / AUTONOMOUS / BLIND_DRIVE)

DOUBLED NAMESPACE — SEE SECTION 6.2

The costmap service paths contain the node name twice. Using the wrong path is a silent no-op. Always use `/global_costmap/global_costmap/set_parameters`, not `/global_costmap/set_parameters`.

zone_nav_manager Parameters

PARAMETER	DEFAULT	UNIT	DESCRIPTION
<code>waypoints_file</code>	<code>zone_waypoints.yaml</code>	path	Path to zone definition file. Absolute or relative to share directory.
<code>total_laps</code>	<code>2</code>	integer	Number of laps before DONE is triggered at finish line. Must be ≥ 1 .
<code>slow_speed_limit</code>	<code>0.2</code>	m/s	Speed limit published during SLOW_NAV zones.
<code>blind_speed</code>	<code>0.15</code>	m/s	Fixed forward speed during BLIND_DRIVE.
<code>blind_timeout_s</code>	<code>15.0</code>	s	Hard timeout in BLIND_DRIVE. Forces exit even if NDT never recovers.
<code>ndt_recovery_threshold</code>	<code>0.8</code>	dimensionless	NDT fitness score below which localization is considered restored. Exit BLIND_DRIVE when fitness < threshold.
<code>speed_reassert_hz</code>	<code>1.0</code>	Hz	Rate at which speed limit is re-published (volatile QoS). Prevents controller_server from reverting to configured max.
<code>push_through_speed</code>	<code>0.25</code>	m/s	Fixed forward speed during PUSH_THROUGH (car wash).

cmd_vel_mux Parameters

PARAMETER	DEFAULT	DESCRIPTION
<code>mode</code>	<code>JOYSTICK</code>	Active mux mode. Set via <code>/cmd_vel_mux_node/set_parameters</code> . Valid: JOYSTICK, AUTONOMOUS, BLIND_DRIVE. ESTOP_LOCK set by hardware only.
<code>joy_topic</code>	<code>/cmd_vel_joy</code>	Joystick velocity input topic name.
<code>nav_topic</code>	<code>/cmd_vel_nav</code>	Nav2 velocity input topic name.
<code>zone_nav_topic</code>	<code>/</code> <code>cmd_vel_zone_nav</code>	zone_nav_manager direct velocity input topic name.
<code>output_topic</code>	<code>/cmd_vel_safe</code>	Arbitrated output to motors.

Zone waypoints are loaded at startup. The file defines a list of named zones. Each zone specifies the centre position, entry/exit radii, and the navigation mode to activate. The parser validates required fields and rejects invalid entries with an error at startup — it does not silently skip them.

```
zones:
  - label: "tunnel_entry"      # string; optional — silently uses "" if omitted
    x: 12.45                  # float; map frame X coordinate (metres); REQUIRED
    y: 4.32                   # float; map frame Y coordinate (metres); REQUIRED
    radius: 0.5               # float (metres); entry trigger distance; REQUIRED
    exit_radius: 0.9          # float (metres); exit trigger distance; REQUIRED
    mode: BLIND_DRIVE         # string; one of the 7 modes; REQUIRED

  - label: "narrow_section"
    x: 7.10
    y: 2.80
    radius: 0.4
    exit_radius: 0.8
    mode: SLOW_NAV

  - label: "finish_line"
    x: 0.50
    y: 0.20
    radius: 0.6
    exit_radius: 1.0
    mode: DONE                # only triggers DONE when lap count == total_laps
```

Field Validation Rules

RULE	ENFORCEMENT	EFFECT OF VIOLATION
<code>mode</code> must be one of the 7 valid modes	Startup validation	Rejected — error logged, zone skipped entirely
<code>x</code> and <code>y</code> are required numeric fields	Startup validation	Rejected — error logged, zone skipped entirely
<code>radius</code> and <code>exit_radius</code> are required	Startup validation	Rejected — error logged, zone skipped entirely
<code>radius</code> must be strictly less than <code>exit_radius</code>	Startup validation	Rejected — equal values cause mode flip-flop; startup error
<code>label</code> field is optional	Silent default	Zone loads with <code>label=""</code> — works but harder to debug
<code>total_laps</code> in manager params must be integer ≥ 1	Startup param check	Defaults to 2 if invalid; warning logged

ZONE ENTRY LOOP SKIPS SAME-MODE ZONES

The zone entry detection loop skips any zone whose `mode == current_mode`. This prevents NORMAL_NAV marker zones from re-triggering NORMAL_NAV and locking `active_zone_idx`. Design implication: do **not** create a `tunnel_exit` zone with `mode=NORMAL_NAV` — it will block other zone triggers within its `exit_radius`.

Key Implementation Gotchas

This section is the most important in the document. It documents the hard-won knowledge distilled from 46 bugs encountered during implementation. Read this before writing any `zone_nav` code.

6.1 Speed Control API

Use `/speed_limit` — Not `SetParameters` on `FollowPath`

- `/speed_limit` topic is the **ONLY** working API for MPPI speed control at runtime
- `FollowPath.vx_max` via `SetParameters` is **silently ignored** during active runs — the controller does not reload params mid-trajectory
- `speed_limit=0.0` means **NO_SPEED_LIMIT** (restores the configured max), NOT "stop the robot". To stop, publish zero `Twist` on `/cmd_vel_zone_nav` with mux in `BLIND_DRIVE` mode.
- controller_server uses **volatile QoS** on `/speed_limit` — the subscription does not use `transient-local`. Solution: `reassertSpeedLimit()` re-sends the limit at 1 Hz so the controller never reverts on a restart or reconnect.

CRITICAL — `SPEED_LIMIT=0.0` IS NOT STOP

Publishing `0.0` on `/speed_limit` removes the speed cap entirely. The robot will accelerate to its configured maximum. To hold position, switch mux to `JOYSTICK` and publish zero `Twist`.

6.2 Costmap Service Paths — The Doubled Namespace

The Namespace Doubling Trap

Nav2 costmap nodes are launched with a namespace and a node name that matches the namespace. This creates a doubled path in the service name. Getting it wrong results in a service call that returns success but does nothing — a completely silent failure.

```
# CORRECT — doubled namespace:
/global_costmap/global_costmap/set_parameters
/local_costmap/local_costmap/set_parameters

# WRONG — single namespace (silent no-op):
/global_costmap/set_parameters
/local_costmap/set_parameters
```

Verify the actual running service names with: `ros2 service list | grep costmap`

6.3 VoxelLayer Limitations

VoxelLayer Has No `min_obstacle_height`

- `VoxelLayer` does **not** support `min_obstacle_height` as a dynamic parameter — only `enabled` bool works dynamically at runtime

- PUSH_THROUGH disables **both** `obstacle_layer.enabled` and `voxel_layer.enabled` via the `costmap SetParameters` services to prevent car-wash ribbons from appearing as a wall
- After PUSH_THROUGH exit, both layers are re-enabled. The costmap will rebuild from fresh lidar scans within one scan cycle (~100ms).

6.4 ESTOP_LOCK is Hardware-Only

● Cannot Set ESTOP_LOCK via ros2 param set

- ESTOP_LOCK mode on `cmd_vel_mux` can **only** be triggered by the `/estop_active` signal going True — it is not settable via `ros2 param set`
- Attempting `ros2 param set /cmd_vel_mux_node mode ESTOP_LOCK` returns an error
- After estop is released (`/estop_active` goes False), the mux reverts to **JOYSTICK** (the safest mode) — the operator must explicitly switch to **AUTONOMOUS** for autonomous navigation to resume

6.5 BLIND_DRIVE Must-Knows

● BLIND_DRIVE Timing and State Details

- The mux is set to BLIND_DRIVE **before** the first `/cmd_vel_zone_nav` is published — there is a 50–100ms gap where the mux switches before the `zone_nav_manager` starts publishing. This is intentional; the mux needs to be ready before the velocity arrives.
- `latest_fitness_` is **reset to 999** on BLIND_DRIVE entry — this prevents the exit condition from triggering immediately on stale pre-entry fitness data that happens to be below the threshold
- `checkFinishLine()` is called **inside the BLIND_DRIVE loop** — if the tunnel is near the finish line, the robot can still detect and respond to the finish line during blind drive
- The **15s hard timeout** triggers if NDT never recovers (e.g., FAST-LIO2 crash). After timeout, the robot returns to **NORMAL_NAV** without lidar-verified position — proceed with caution.

6.6 zone_waypoints.yaml Pitfalls

● Common YAML Configuration Mistakes

- **Do NOT add a tunnel_exit zone with mode=NORMAL_NAV** — it sets `active_zone_idx_` and blocks all other zone triggers within `exit_radius` for as long as the robot remains inside that radius
- The zone entry loop **skips zones whose mode == current_mode_** — this is a feature, not a bug. It prevents **NORMAL_NAV** marker zones from re-locking zone detection. Side effect: you can't chain two same-mode zones that are close together.
- **radius must be strictly less than exit_radius** — hysteresis is required. Equal values cause repeated mode flip-flop as the robot hovers at the boundary. The parser rejects equal values at startup.
- An **unlabeled zone** (no `label :` field) loads silently with `label=""` — no warning. This makes debug logs harder to read.
- A **zone without mode:** is rejected at startup with an error — it does not silently default.

6.7 lidar_odom_gate Must Always Run with use_localization

● Gate Node Is Not Optional

- EKF2 subscribes to `/lidar_odometry_gated` — the gate node's output. **Without the gate node running, EKF2 receives no lidar odometry at all**, even when `zone_nav` is in `NORMAL_NAV`.
- The gate has a **5s watchdog**: if `zone_nav_manager` crashes while in `BLIND_DRIVE` (gate closed), the gate automatically re-opens after 5s of silence on `/nav_mode`. This prevents EKF2 from starving permanently if the manager crashes mid-tunnel.
- Default mode is **INIT**, which keeps the gate open (pass-through). Safe for development runs without `zone_nav` active.

Step-by-step diagnostics. Work top-to-bottom. Each step assumes the previous passes.

Topics to Monitor During a Test Run

● Monitor Commands

```
# State machine state (expect INIT until green light)
ros2 topic echo /nav_mode

# Mux arbitration (expect AUTONOMOUS during Nav2 run)
ros2 topic echo /mux_mode

# NDT quality: >0.8 in tunnel, <0.8 after exit
ros2 topic echo /ndt_fitness_score

# Speed cap active
ros2 topic echo /speed_limit

# What actually reaches the motors
ros2 topic echo /cmd_vel_safe

# Must be False for robot to move
ros2 topic echo /estop_active

# Robot position in map frame
ros2 topic echo /odometry/filtered --field pose.pose.position
```

● Service Verification Commands

```
# Verify costmap services exist (doubled namespace)
ros2 service list | grep global_costmap
ros2 service list | grep local_costmap
ros2 service list | grep cmd_vel_mux_node

# Manually trigger green light (bench test)
ros2 topic pub --once /green_light std_msgs/msg/Bool "data: true"

# Check current mux mode
ros2 param get /cmd_vel_mux_node mode

# Force AUTONOMOUS mode manually
ros2 param set /cmd_vel_mux_node mode AUTONOMOUS

# Check node list
ros2 node list | grep -E "zone_nav|cmd_vel_mux|lidar_odom_gate"
```

Common Failure Modes and Diagnostics

SYMPTOM	FIRST CHECK	LIKELY CAUSE	FIX
Robot won't move after green light	<code>ros2 topic echo / mux_mode</code>	Mux still in JOYSTICK, or <code>/ estop_active</code> is True	Check <code>/estop_active</code> . Manually set mux to AUTONOMOUS if estop clear.
Robot ignores zone entry	<code>ros2 topic echo / nav_mode</code>	Zone coordinates wrong vs actual robot position, or zone has same mode as current	Echo <code>/odometry/filtered</code> near zone. Compare against YAML coordinates. Check mode != current_mode_rule.
Speed doesn't change in zone	<code>ros2 topic echo / speed_limit</code>	speed_limit not publishing, or controller_server not running	Check zone_nav_manager is alive. Verify controller_server node is up with <code>ros2 node list</code> .
Costmap not updating in PUSH_THROUGH	<code>ros2 service list grep costmap</code>	Wrong service path (single namespace instead of doubled)	Verify service path has doubled namespace. See Section 6.2.
Robot drifts severely in tunnel	<code>ros2 topic echo / lidar_odometry_gated</code>	Gate not closing — lidar still feeding EKF2 with corrupt odometry during BLIND_DRIVE	Confirm lidar_odom_gate node is running. Check /nav_mode is publishing BLIND_DRIVE. Check gate node logs.
DONE never triggered at finish	Check finish_line coordinates in YAML	Finish zone coordinates wrong, or lap counter never reached total_laps	Echo <code>/odometry/filtered</code> at finish. Verify <code>/ndt_fitness_score</code> recovered after tunnel (needed for position accuracy).
EKF2 has no lidar input	<code>ros2 topic hz / lidar_odometry_gated</code>	lidar_odom_gate not running, or gate stuck closed (zone_nav crashed in BLIND_DRIVE)	Start lidar_odom_gate. If crashed mid-tunnel, gate re-opens after 5s watchdog — or restart gate node.

QUICK HEALTH CHECK

Run `scripts/health_check_zone_nav.sh` before every test run. It checks all topics, services, and current mode values in under 30 seconds and gives you a PASS/FAIL summary.

All scripts live in `scripts/` at the repository root. Run `chmod +x scripts/*.sh` once after cloning. All scripts require a sourced ROS 2 environment.

● `health_check_zone_nav.sh`

Pre-run sanity check. Run before every test. Checks all required topics are publishing (`/nav_mode`, `/mux_mode`, `/odometry/filtered`, `/ndt_fitness_score`, `/cmd_vel_safe`). Verifies all three SetParameters services are available. Reads current mode values. Prints colour-coded PASS/FAIL for each check. Exits 0 if all pass, 1 if any fail.

```
./scripts/health_check_zone_nav.sh
```

● `inspect_zone_nav.sh`

Live dashboard. Updates every 1 second. Shows current state machine mode, mux mode, e-stop status, NDT fitness score, robot X/Y position, active speed limit, and current `/cmd_vel_safe` output. Handles timeouts gracefully (shows TIMEOUT in red). Press Ctrl-C to exit.

```
./scripts/inspect_zone_nav.sh
```

● `trigger_green_light.sh`

Bench testing without the physical green light. Publishes a single True message on `/green_light` to trigger the INIT → NORMAL_NAV transition. Use this when developing on a bench without the physical starting light hardware connected.

```
./scripts/trigger_green_light.sh
```

● `set_mux_mode.sh`

Manual mode switch helper. Calls `ros2 param set` on `/cmd_vel_mux_node` to change mux mode. Accepts JOYSTICK, AUTONOMOUS, or BLIND_DRIVE. Refuses ESTOP_LOCK (hardware-only) with an explanatory error. Prints current mode if called with no argument.

```
./scripts/set_mux_mode.sh AUTONOMOUS  
./scripts/set_mux_mode.sh JOYSTICK
```


RECOMMENDED TEST RUN ORDER

1. Source ROS 2 workspace. 2. Launch full stack. 3. Run `health_check_zone_nav.sh` — fix any FAILs. 4. Open `inspect_zone_nav.sh` in a second terminal. 5. Run `trigger_green_light.sh`. 6. Watch inspect dashboard: `nav_mode` should change INIT → NORMAL_NAV, `mux_mode` → AUTONOMOUS.